Learning to Rank

NDCG and a pairwise ranker

The problem

- Query → many candidate documents
- Order them by relevance
- Top positions matter most

DCG and NDCG

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$NDCG = \frac{DCG}{IDCG}$$

- Gain grows with relevance
- Discount shrinks with position
- Normalise by the ideal order → [0, 1]

NDCG

```
def dcg(rels, k=None):
    rels = np.asarray(rels, float)[:k]
    return float(np.sum((2 ** rels - 1) / np.log2(np.arange(2, rels.size + 2))))

def ndcg(rels_in_ranked_order, k=None):
    ideal = dcg(sorted(rels_in_ranked_order, reverse=True), k)
    return dcg(rels_in_ranked_order, k) / ideal if ideal > 0 else 0.0
```

Pairwise approach

$$L = \sum_{(i,\,j)\,:\,rel_i > rel_j} \log\left(1 + e^{-(s_i - s_j)}\right)$$

- Turn ranking into "is i above j?" decisions
- Only compare documents of the same query
- Logistic loss on score differences

PairwiseRanker.fit

```
def fit(self, X, rel, qid):
    X, rel, qid = np.asarray(X, float), np.asarray(rel), np.asarray(qid)
    pairs = [(i, j) for q in np.unique(qid) for i in np.where(qid == q)[0] for j in np.where(qid == q)[0] if rel[i] > rel[j]]
    self.w = np.zeros(X.shape[1])
    if not pairs:
        return self
    I, J = np.array(pairs).T
    D = X[I] - X[J]
    for _ in range(self.epochs):
        p = 1 / (1 + np.exp(D @ self.w))  # = sigmoid(-(s_i - s_j))
        self.w += self.lr * (D * p[:, None]).mean(0)
    return self
```

Trade-offs

- Pointwise: regress relevance — simple, ignores order
- Pairwise: $O(m^2)$ pairs per query
- Listwise (LambdaMART): optimise NDCG-like objectives
- Watch position bias in click labels

Recap

- Evaluate with NDCG@k
- Pairwise: logistic loss on score differences
- Pairs only within a query
- Listwise methods for production